

Here is a **complete, easy, exam-oriented overview of Maven** with **definitions, architecture, lifecycle, commands, POM, dependencies, plugins, interview questions, and MCQs.**

✅ MAVEN – Complete Overview

📌 What is Maven?

Maven is a **build automation and project management tool** used mainly in Java projects.

It handles:

- **Build** (compile → test → package)
 - **Dependency management**
 - **Project structure**
 - **Plugins**
 - **Project documentation**
-

🌳 📁 Maven Standard Directory Structure

CSS

📄 Copy code

```
project-name/
├── src/
│   ├── main/
│   │   ├── java/      → Application code
│   │   ├── resources/ → Config files, properties
│   │   └── test/
│   │       ├── java/   → Test cases
│   └── pom.xml         → Project Object Model
```

📦 POM.XML (Very Important)

POM = Project Object Model

This file contains:

- Project details
- Dependencies
- Plugins
- Build info

Example POM

xml

 Copy code

```
<project>
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.yuvraj</groupId>
  <artifactId>maven-demo</artifactId>
  <version>1.0-SNAPSHOT</version>

  <dependencies>
    <dependency>
      <groupId>mysql</groupId>
      <artifactId>mysql-connector-java</artifactId>
      <version>8.0.29</version>
    </dependency>
  </dependencies>
</project>
```



Maven Repository Types

Repository	Description
Local Repo	On your system (.m2 folder)
Central Repo	Online Maven repository
Remote Repo	Company/Custom servers



Maven Build Lifecycle

Maven has 3 main lifecycles:

1 Clean Lifecycle

Phase	Function
pre-clean	Before clean
clean	Deletes target directory
post-clean	After clean

2 Default Lifecycle (Most important)

Phase	Description
validate	Validate project
compile	Compile source code
test	Run test cases
package	Create JAR/WAR
verify	Integration tests
install	Install in local repo
deploy	Upload to remote repo

3 Site Lifecycle

Phase	Description
site	Generate documentation

Most Used Maven Commands

Command	Description
mvn clean	Delete target folder
mvn compile	Compile code
mvn test	Run tests
mvn package	Build jar/war
mvn install	Add to local repo
mvn deploy	Deploy to remote repo
mvn clean install	Clean + compile + package + install

Dependencies (Very Important)

In Maven, dependency means **external jar file** added using POM.

Example: MySQL Driver

xml

 Copy code

```
<dependency>
  <groupId>mysql</groupId>
  <artifactId>mysql-connector-java</artifactId>
  <version>8.0.30</version>
</dependency>
```

Maven Plugins

Plugins are used for additional tasks like packaging WAR, running tests, generating reports.

Example: Compiler Plugin

xml

 Copy code

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
```

```
<artifactId>maven-compiler-plugin</artifactId>
<version>3.8.1</version>
<configuration>
  <source>1.8</source>
  <target>1.8</target>
</configuration>
</plugin>
```

Why Use Maven? (Advantages)

- Automatic dependency management
 - Standard directory structure
 - Faster project setup
 - Easy build & deployment
 - Works well with Jenkins, Spring Boot, Hibernate, etc.
-

Interview Questions (Advanced)

1. Difference between Maven and Gradle?

Maven	Gradle
XML (POM)	Groovy/Kotlin
Slower	Faster
Lifecycle-based	Task-based

2. What is transitive dependency?

If Dependency A requires B, and B requires C, Maven downloads C automatically.

3. What is groupId, artifactId, version?

- **groupId** → Company name (unique)
- **artifactId** → Project name
- **version** → Project version

4. What is dependency scope?

Scope	Meaning
compile	Available everywhere
provided	Provided by server (Tomcat)
runtime	Needed only during execution
test	Only for testing

5. How to skip tests?

nginx

 Copy code

```
mvn install -DskipTests
```



40 MCQs on Maven (With Answers)

(If you want full 100 MCQs, tell me)

1. Maven is written in:

- A. C++
- B. Java
- C. Python
- D. Ruby

Answer: B

2. POM stands for:

- A. Project Object Model

Answer: A

3. Default POM filename:

- A. project.xml
- B. build.xml

C. pom.xml

Answer: C

4. Which command creates jar/war?

A. mvn clean

B. mvn compile

C. mvn package

Answer: C

5. Local repository location?

A. /etc/m2

B. .m2/repository

Answer: B

If you want:

 **Maven mini project**

 **100 MCQ with explanations**

 **Advanced Maven interview Q&A**

 **Spring Boot with Maven setup**

Just say “Next”.

How to create Maven project on eclipse

Here is a **simple, step-by-step guide** to create a **Maven project in Eclipse** (100% correct for exams & practicals).

How to Create a Maven Project in Eclipse

Step 1: Open Eclipse

Open your Eclipse IDE (preferably Eclipse IDE for Java EE).

Step 2: Install Maven Plugin (If Not Installed)

Most Eclipse versions already have Maven (m2e) pre-installed.

Check:

```
css
```

 Copy code

```
Help → About Eclipse → Installation Details  
Search for "m2e" or "Maven Integration"
```

If not installed:

```
sql
```

 Copy code

```
Help → Install New Software → Work with:  
https://download.eclipse.org/releases/latest  
Select "Maven Integration"
```

Step 3: Create New Maven Project

1. Go to:

```
arduino
```

 Copy code

```
File → New → Project
```

2. Select:

```
nginx
```

 Copy code

```
Maven → Maven Project
```

3. Click **Next**.

Step 4: Project Location

You will see:

✓ Create a simple project (skip archetype selection)

✗ (Do not select this)

Instead:

- **Keep checkbox unchecked**
- Click **Next**

Step 5: Select Archetype

You will now select a project template.

Common archetype:

 Copy code

```
maven-archetype-quickstart
```

If not found:

- In the catalog list → select **All Catalogs**

Then choose:

maven-archetype-quickstart (Java SE Project)

Click **Next**.

Step 6: Enter Project Details

You must fill:

Field	Meaning
Group Id	Your company/project domain (e.g., <code>com.yuvraj</code>)
Artifact Id	Project name (e.g., <code>mavenDemo</code>)
Version	Default (1.0-SNAPSHOT)

Example:

```
yaml
```

 Copy code

```
Group Id:    com.yuvraj
```

```
Artifact Id: mavenDemo
```

Click **Finish**.

Step 7: Maven Project Structure Appears

Eclipse creates:

CSS

 Copy code

mavenDemo

- |— src/main/java
- |— src/test/java
- |— JRE System Library
- |— Maven Dependencies
- |— pom.xml

Step 8: Update Project (Important)

Right-click on the project →

sql

 Copy code

Maven → **Update** Project → OK

This downloads required dependencies into .m2/repository .

Step 9: Add Dependency in POM (Example – MySQL)

Open pom.xml → add:

xml

 Copy code

```
<dependency>
  <groupId>mysql</groupId>
  <artifactId>mysql-connector-java</artifactId>
  <version>8.0.30</version>
</dependency>
```

Then update project again:

Right-click → Maven → Update Project

 **Your Maven project is successfully created!**

 **Do you want?**

- ✓ **Maven project in Eclipse with Servlets**
- ✓ **Maven + Spring Boot project setup**
- ✓ **Maven project screenshots**
- ✓ **Create WAR file using Maven in Eclipse**
- ✓ **How to run Maven project**

Just type **Next**.

Maven project in eclipse with servlet

Below is a **complete, step-by-step guide** to create a **Maven Project in Eclipse with Servlets** — including configuration, folder structure, dependencies, and how to run on Tomcat.

 **Create Maven Project in Eclipse (Servlet + JSP Ready)**

 **Step 1: Create Maven Project**

1. Open Eclipse
2. Go to:

File → New → Project → Maven → Maven Project

3. Click **Next**
4. Uncheck “Create a simple project (skip archetype selection)”
5. Click **Next**

✓ Step 2: Select Web Application Archetype

Search for:

 Copy code

```
maven-archetype-webapp
```

Select:

 Copy code

```
org.apache.maven.archetypes → maven-archetype-webapp
```

Click **Next**

✓ Step 3: Enter Project Details

Example:

```
makefile
```

 Copy code

```
GroupId:      com.yuvraj
ArtifactId:   ServletMavenApp
Version:      1.0-SNAPSHOT
```

Click **Finish**

📁 Step 4: Project Structure

After creation, Eclipse will show:

```
CSS
```

 Copy code

```
ServletMavenApp
├── src/main/webapp
│   └── WEB-INF
```

```
|      |      | web.xml
|      |      | index.jsp
|----- src/main/java      ← (Create manually)
|----- pom.xml
```

👉 **Important:** Maven webapp archetype does NOT create `src/main/java` .
Create it manually:

CSS

 Copy code

Right click → `src/main` → New → Folder → `src/main/java`

✅ Step 5: Add Servlet API Dependency

Open `pom.xml` and add:

xml

 Copy code

```
<dependencies>
  <dependency>
    <groupId>javax.servlet</groupId>
    <artifactId>javax.servlet-api</artifactId>
    <version>4.0.1</version>
    <scope>provided</scope>
  </dependency>
</dependencies>
```

💡 `provided` means: the JAR is provided by **Tomcat** at runtime.

↺ Step 6: Update Maven Project

Right-click project →

sql

 Copy code

Maven → **Update** Project → OK

🔧 Step 7: Create Your Servlet

Create package and servlet:

CSS

 Copy code

```
src/main/java
  com.yuvraj.servlet
    MyServlet.java
```

MyServlet.java

java

 Copy code

```
package com.yuvraj.servlet;

import java.io.IOException;
import javax.servlet.*;
import javax.servlet.http.*;

public class MyServlet extends HttpServlet {

    protected void doGet(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {
        resp.setContentType("text/html");
        resp.getWriter().println("<h1>Hello from Maven Servlet!</h1>");
    }
}
```

Step 8: Configure web.xml

Open:

bash

 Copy code

```
src/main/webapp/WEB-INF/web.xml
```

Add servlet mapping:

xml

 Copy code

```
<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee"
  version="4.0">
```

```
<servlet>
  <servlet-name>MyServlet</servlet-name>
  <servlet-class>com.yuvraj.servlet.MyServlet</servlet-class>
</servlet>

<servlet-mapping>
  <servlet-name>MyServlet</servlet-name>
  <url-pattern>/hello</url-pattern>
</servlet-mapping>

</web-app>
```

Step 9: Add Tomcat to Eclipse

1. Go to:

pgsql

 Copy code

Window → Preferences → Server → Runtime Environments

2. Add:

nginx

 Copy code

Apache Tomcat 9 / 10

3. Select Tomcat installation folder.

Step 10: Run the Project

1. Right-click on project
2. Select:

pgsql

 Copy code

Run As → Run on Server

3. Choose Tomcat server → Finish

Step 11: Test Servlet

Open browser:

```
bash
```

 Copy code

```
http://localhost:8080/ServletMavenApp/hello
```

You will see:

```
csharp
```

 Copy code

```
Hello from Maven Servlet!
```

Bonus (Recommended)

I can also give you:

- ✓ Complete Maven + Servlet Login / Registration Project
- ✓ Maven + JSP + JDBC CRUD project
- ✓ Full folder structure + code
- ✓ WAR file creation and deployment guide
- ✓ Error fixing (404, missing web.xml, Tomcat config)

Just type **Next**.

Complete Maven servlet login registration project

Below is a **complete Maven + Servlet + JSP + JDBC Login & Registration Project** with:

- ✓ Full folder structure
- ✓ pom.xml
- ✓ Database (users table)
- ✓ Registration servlet
- ✓ Login servlet
- ✓ DAO + DB connection
- ✓ JSP pages

- ✓ web.xml
- ✓ Perfect for college project / viva

PROJECT NAME: MavenLoginRegister

1. Project Folder Structure (Maven Standard)

CSS

 Copy code

MavenLoginRegister

```
|— src/main/java
|   |— com.yuvraj
|       |— dao
|           |— UserDAO.java
|       |— model
|           |— User.java
|       |— servlet
|           |— RegisterServlet.java
|           |— LoginServlet.java
|       |— util
|           |— DBConnection.java
|
|— src/main/webapp
|   |— register.jsp
|   |— login.jsp
|   |— welcome.jsp
|   |— error.jsp
|   |— WEB-INF
|       |— web.xml
|
|— pom.xml
```

2. MySQL Database Setup

Create DB:

sql

 Copy code

```
CREATE DATABASE mavenlogin;  
USE mavenlogin;
```

Create table:

sql

 Copy code

```
CREATE TABLE users (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(100),  
    email VARCHAR(100) UNIQUE,  
    password VARCHAR(100)  
);
```

3. pom.xml

Add MySQL + Servlet dependencies.

xml

 Copy code

```
<project xmlns="http://maven.apache.org/POM/4.0.0"  
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0  
        http://maven.apache.org/xsd/maven-4.0.0.xsd">  
  
    <modelVersion>4.0.0</modelVersion>  
  
    <groupId>com.yuvraj</groupId>  
    <artifactId>MavenLoginRegister</artifactId>  
    <version>1.0-SNAPSHOT</version>  
    <packaging>war</packaging>  
  
    <dependencies>  
  
        <!-- Servlet API -->  
        <dependency>  
            <groupId>javax.servlet</groupId>  
            <artifactId>javax.servlet-api</artifactId>  
            <version>4.0.1</version>  
            <scope>provided</scope>  
        </dependency>
```

```
<!-- MySQL JDBC -->
<dependency>
  <groupId>mysql</groupId>
  <artifactId>mysql-connector-java</artifactId>
  <version>8.0.33</version>
</dependency>

<!-- JSP Support -->
<dependency>
  <groupId>javax.servlet.jsp</groupId>
  <artifactId>javax.servlet.jsp-api</artifactId>
  <version>2.3.3</version>
  <scope>provided</scope>
</dependency>

</dependencies>
</project>
```

4. DBConnection.java

java

 Copy code

```
package com.yuvraj.util;

import java.sql.Connection;
import java.sql.DriverManager;

public class DBConnection {
    private static final String URL = "jdbc:mysql://localhost:3306/mavenlog";
    private static final String USER = "root";
    private static final String PASS = "root"; // change if needed

    public static Connection getConnection() {
        Connection con = null;
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            con = DriverManager.getConnection(URL, USER, PASS);
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

```
        return con;
    }
}
```

5. User.java (Model Class)

java

 Copy code

```
package com.yuvraj.model;

public class User {
    private int id;
    private String name;
    private String email;
    private String password;

    public User() {}

    public User(String name, String email, String password) {
        this.name = name;
        this.email = email;
        this.password = password;
    }

    // Getters & Setters
    public int getId() { return id; }
    public void setId(int id) { this.id = id; }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }

    public String getPassword() { return password; }
    public void setPassword(String password) { this.password = password; }
}
```

6. UserDAO.java

```
package com.yuvraj.dao;

import com.yuvraj.model.User;
import com.yuvraj.util.DBConnection;

import java.sql.*;

public class UserDAO {

    public boolean register(User user) {
        String sql = "INSERT INTO users(name, email, password) VALUES (?, ?)";
        try (Connection con = DBConnection.getConnection();
            PreparedStatement ps = con.prepareStatement(sql)) {

            ps.setString(1, user.getName());
            ps.setString(2, user.getEmail());
            ps.setString(3, user.getPassword());

            return ps.executeUpdate() > 0;

        } catch (Exception e) {
            e.printStackTrace();
        }
        return false;
    }

    public boolean login(String email, String password) {
        String sql = "SELECT * FROM users WHERE email=? AND password=?";
        try (Connection con = DBConnection.getConnection();
            PreparedStatement ps = con.prepareStatement(sql)) {

            ps.setString(1, email);
            ps.setString(2, password);

            ResultSet rs = ps.executeQuery();
            return rs.next();

        } catch (Exception e) {
            e.printStackTrace();
        }
        return false;
    }
}
```

```
}  
}
```

7. RegisterServlet.java

java

 Copy code

```
package com.yuvraj.servlet;  
  
import com.yuvraj.dao.UserDAO;  
import com.yuvraj.model.User;  
  
import javax.servlet.*;  
import javax.servlet.http.*;  
import java.io.IOException;  
  
public class RegisterServlet extends HttpServlet {  
  
    protected void doPost(HttpServletRequest req, HttpServletResponse resp)  
        throws ServletException, IOException {  
  
        String name = req.getParameter("name");  
        String email = req.getParameter("email");  
        String pass = req.getParameter("password");  
  
        User user = new User(name, email, pass);  
        UserDAO dao = new UserDAO();  
  
        if (dao.register(user)) {  
            resp.sendRedirect("login.jsp");  
        } else {  
            resp.sendRedirect("error.jsp");  
        }  
    }  
}
```

8. LoginServlet.java

java

 Copy code

```

package com.yuvraj.servlet;

import com.yuvraj.dao.UserDAO;

import javax.servlet.*;
import javax.servlet.http.*;
import java.io.IOException;

public class LoginServlet extends HttpServlet {

    protected void doPost(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {

        String email = req.getParameter("email");
        String pass = req.getParameter("password");

        UserDAO dao = new UserDAO();

        if (dao.login(email, pass)) {
            HttpSession session = req.getSession();
            session.setAttribute("email", email);
            resp.sendRedirect("welcome.jsp");
        } else {
            resp.sendRedirect("error.jsp");
        }
    }
}

```

9. web.xml

xml

 Copy code

```

<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee"
    version="4.0">

    <servlet>
        <servlet-name>RegisterServlet</servlet-name>
        <servlet-class>com.yuvraj.servlet.RegisterServlet</servlet-class>
    </servlet>

    <servlet-mapping>

```

```
<servlet-name>RegisterServlet</servlet-name>
<url-pattern>/register</url-pattern>
</servlet-mapping>

<servlet>
  <servlet-name>LoginServlet</servlet-name>
  <servlet-class>com.yuvraj.servlet.LoginServlet</servlet-class>
</servlet>

<servlet-mapping>
  <servlet-name>LoginServlet</servlet-name>
  <url-pattern>/login</url-pattern>
</servlet-mapping>

</web-app>
```

10. JSP Pages

register.jsp

jsp

 Copy code

```
<form action="register" method="post">
  <h2>Register</h2>
  Name: <input type="text" name="name" required><br>
  Email: <input type="email" name="email" required><br>
  Password: <input type="password" name="password" required><br>
  <button type="submit">Register</button>
</form>
```

login.jsp

jsp

 Copy code

```
<form action="login" method="post">
  <h2>Login</h2>
  Email: <input type="email" name="email" required><br>
  Password: <input type="password" name="password" required><br>
```



```
<button type="submit">Login</button>
</form>
```

welcome.jsp

jsp

 Copy code

```
<%
String email = (String) session.getAttribute("email");
if(email == null){
    response.sendRedirect("login.jsp");
}
%>

<h2>Welcome, <%= email %></h2>
<a href="logout.jsp">Logout</a>
```

error.jsp

jsp

 Copy code

```
<h2>Something went wrong!</h2>
<a href="login.jsp">Try Again</a>
```

NEXT (Optional but Recommended)

I can also give you:

- ✓ Logout servlet + session destroy
- ✓ SHA-256 password encryption (more secure)
- ✓ Bootstrap UI for JSP pages
- ✓ Home page + Navbar
- ✓ WAR file creation + Tomcat deployment
- ✓ Video explanation of flowchart

Just type **Next**.

Maven jsp jdbc crud project

Below is a **complete Maven + JSP + Servlet + JDBC CRUD Project** with:

- ✓ Create
- ✓ Read
- ✓ Update
- ✓ Delete

Perfect for internship, viva, and B.E. final-year project.



PROJECT NAME: MavenJSPJDBCCRUD



Database: MySQL

sql

Copy code

```
CREATE DATABASE cruddb;
USE cruddb;

CREATE TABLE users (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(100),
  email VARCHAR(100),
  country VARCHAR(100)
);
```



Project Folder Structure (Maven Standard)

pgsql

Copy code

```
MavenJSPJDBCCRUD
├─ src/main/java
```

```
|      └─ com.project
|          │   └─ dao
|          │       └─ UserDAO.java
|          │   └─ model
|          │       └─ User.java
|          └─ servlet
|              │   └─ AddUserServlet.java
|              │   └─ UpdateUserServlet.java
|              │   └─ DeleteUserServlet.java
|              └─ EditUserServlet.java
|
| └─ src/main/webapp
|     │   └─ add-user.jsp
|     │   └─ edit-user.jsp
|     │   └─ list-users.jsp
|     └─ WEB-INF
|         └─ web.xml
|
└─ pom.xml
```

pom.xml

xml

 Copy code

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <groupId>com.project</groupId>
  <artifactId>MavenJSPJDBCCRUD</artifactId>
  <version>1.0-SNAPSHOT</version>
  <packaging>war</packaging>

  <dependencies>

    <!-- Servlet API -->
    <dependency>
      <groupId>javax.servlet</groupId>
```

```
        <artifactId>javax.servlet-api</artifactId>
        <version>4.0.1</version>
        <scope>provided</scope>
    </dependency>

    <!-- JSP Support -->
    <dependency>
        <groupId>javax.servlet.jsp</groupId>
        <artifactId>javax.servlet.jsp-api</artifactId>
        <version>2.3.3</version>
        <scope>provided</scope>
    </dependency>

    <!-- MySQL JDBC -->
    <dependency>
        <groupId>mysql</groupId>
        <artifactId>mysql-connector-java</artifactId>
        <version>8.0.33</version>
    </dependency>

</dependencies>

</project>
```

DBConnection.java

java

 Copy code

```
package com.project.dao;

import java.sql.*;

public class DBConnection {

    private static final String URL = "jdbc:mysql://localhost:3306/cruddb"
    private static final String USER = "root";
    private static final String PASS = "root";

    public static Connection getConnection() {
        Connection con = null;
        try {
```

```
        Class.forName("com.mysql.cj.jdbc.Driver");
        con = DriverManager.getConnection(URL, USER, PASS);
    } catch (Exception e) {
        e.printStackTrace();
    }
    return con;
}
}
```



User.java (MODEL)

java

 Copy code

```
package com.project.model;

public class User {
    private int id;
    private String name;
    private String email;
    private String country;

    public User() {}

    public User(String name, String email, String country) {
        this.name = name;
        this.email = email;
        this.country = country;
    }

    public User(int id, String name, String email, String country) {
        this.id = id;
        this.name = name;
        this.email = email;
        this.country = country;
    }

    // Getters and setters...
}
```

UserDAO.java (CRUD Logic)

java

 Copy code

```
package com.project.dao;

import com.project.model.User;
import java.sql.*;
import java.util.*;

public class UserDAO {

    // INSERT
    public void addUser(User user) {
        String sql = "INSERT INTO users(name,email,country) VALUES (?,?,?)"
        try (Connection con = DBConnection.getConnection();
            PreparedStatement ps = con.prepareStatement(sql)) {

            ps.setString(1, user.getName());
            ps.setString(2, user.getEmail());
            ps.setString(3, user.getCountry());
            ps.executeUpdate();

        } catch (Exception e) {
            e.printStackTrace();
        }
    }

    // SELECT ALL
    public List<User> getAllUsers() {
        List<User> list = new ArrayList<>();
        try (Connection con = DBConnection.getConnection();
            PreparedStatement ps = con.prepareStatement("SELECT * FROM users")) {

            ResultSet rs = ps.executeQuery();
            while (rs.next()) {
                list.add(new User(
                    rs.getInt("id"),
                    rs.getString("name"),
                    rs.getString("email"),
                    rs.getString("country"))
                );
            }
        }
    }
}
```

```

    } catch (Exception e) {
        e.printStackTrace();
    }
    return list;
}

// SELECT BY ID
public User getUserById(int id) {
    User user = null;
    try (Connection con = DBConnection.getConnection();
        PreparedStatement ps = con.prepareStatement("SELECT * FROM users WHERE id = ?")) {

        ps.setInt(1, id);
        ResultSet rs = ps.executeQuery();

        if (rs.next()) {
            user = new User(
                rs.getInt("id"),
                rs.getString("name"),
                rs.getString("email"),
                rs.getString("country")
            );
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
    return user;
}

// UPDATE
public void updateUser(User user) {
    String sql = "UPDATE users SET name=?, email=?, country=? WHERE id=?";
    try (Connection con = DBConnection.getConnection();
        PreparedStatement ps = con.prepareStatement(sql)) {

        ps.setString(1, user.getName());
        ps.setString(2, user.getEmail());
        ps.setString(3, user.getCountry());
        ps.setInt(4, user.getId());
        ps.executeUpdate();

    } catch (Exception e) {
        e.printStackTrace();
    }
}

```

```

    }
}

// DELETE
public void deleteUser(int id) {
    try (Connection con = DBConnection.getConnection();
        PreparedStatement ps = con.prepareStatement("DELETE FROM users")) {

        ps.setInt(1, id);
        ps.executeUpdate();

    } catch (Exception e) {
        e.printStackTrace();
    }
}
}

```

SERVLETS

AddUserServlet.java

java

 Copy code

```

package com.project.servlet;

import com.project.dao.UserDAO;
import com.project.model.User;

import javax.servlet.*;
import javax.servlet.http.*;
import java.io.IOException;

public class AddUserServlet extends HttpServlet {

    protected void doPost(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {

        String name = req.getParameter("name");
        String email = req.getParameter("email");
        String country = req.getParameter("country");
    }
}

```



```
User user = new User(name, email, country);
new UserDAO().addUser(user);

resp.sendRedirect("list-users.jsp");
}
}
```

EditUserServlet.java

java

 Copy code

```
package com.project.servlet;

import com.project.dao.UserDAO;
import com.project.model.User;

import javax.servlet.*;
import javax.servlet.http.*;
import java.io.IOException;

public class EditUserServlet extends HttpServlet {

    protected void doGet(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {

        int id = Integer.parseInt(req.getParameter("id"));
        User user = new UserDAO().getUserById(id);

        req.setAttribute("user", user);
        RequestDispatcher rd = req.getRequestDispatcher("edit-user.jsp");
        rd.forward(req, resp);
    }
}
```

UpdateUserServlet.java

java

 Copy code

```
package com.project.servlet;

import com.project.dao.UserDAO;
import com.project.model.User;

import javax.servlet.*;
import javax.servlet.http.*;
import java.io.IOException;

public class UpdateUserServlet extends HttpServlet {

    protected void doPost(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {

        int id = Integer.parseInt(req.getParameter("id"));
        String name = req.getParameter("name");
        String email = req.getParameter("email");
        String country = req.getParameter("country");

        User user = new User(id, name, email, country);
        new UserDAO().updateUser(user);

        resp.sendRedirect("list-users.jsp");
    }
}
```

DeleteUserServlet.java

java

 Copy code

```
package com.project.servlet;

import com.project.dao.UserDAO;

import javax.servlet.*;
import javax.servlet.http.*;
import java.io.IOException;

public class DeleteUserServlet extends HttpServlet {

    protected void doGet(HttpServletRequest req, HttpServletResponse resp)
```

```

        throws ServletException, IOException {

    int id = Integer.parseInt(req.getParameter("id"));
    new UserDAO().deleteUser(id);

    resp.sendRedirect("list-users.jsp");
}
}

```

web.xml

xml

 Copy code

```

<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee" version="4.0">

    <servlet>
        <servlet-name>AddUserServlet</servlet-name>
        <servlet-class>com.project.servlet.AddUserServlet</servlet-class>
    </servlet>
    <servlet-mapping>
        <url-pattern>/addUser</url-pattern>
        <servlet-name>AddUserServlet</servlet-name>
    </servlet-mapping>

    <servlet>
        <servlet-name>EditUserServlet</servlet-name>
        <servlet-class>com.project.servlet.EditUserServlet</servlet-class>
    </servlet>
    <servlet-mapping>
        <url-pattern>/editUser</url-pattern>
        <servlet-name>EditUserServlet</servlet-name>
    </servlet-mapping>

    <servlet>
        <servlet-name>UpdateUserServlet</servlet-name>
        <servlet-class>com.project.servlet.UpdateUserServlet</servlet-class>
    </servlet>
    <servlet-mapping>
        <url-pattern>/updateUser</url-pattern>
        <servlet-name>UpdateUserServlet</servlet-name>
    </servlet-mapping>

```

```
<servlet>
  <servlet-name>DeleteUserServlet</servlet-name>
  <servlet-class>com.project.servlet.DeleteUserServlet</servlet-class>
</servlet>
<servlet-mapping>
  <url-pattern>/deleteUser</url-pattern>
  <servlet-name>DeleteUserServlet</servlet-name>
</servlet-mapping>

</web-app>
```

JSP Pages

add-user.jsp

jsp

 Copy code

```
<h2>Add New User</h2>
<form action="addUser" method="post">
  Name: <input type="text" name="name"><br>
  Email: <input type="text" name="email"><br>
  Country: <input type="text" name="country"><br>
  <button type="submit">Save</button>
</form>
```

edit-user.jsp

jsp

 Copy code

```
<%@ page import="com.project.model.User" %>
<%
User user = (User) request.getAttribute("user");
%>

<h2>Edit User</h2>
<form action="updateUser" method="post">
  <input type="hidden" name="id" value="<%= user.getId() %>" />
```

```
Name: <input type="text" name="name" value="<%= user.getName() %>"/><br>
Email: <input type="text" name="email" value="<%= user.getEmail() %>"/>
Country: <input type="text" name="country" value="<%= user.getCountry() %>"/>
<button type="submit">Update</button>
</form>
```

list-users.jsp

jsp

 Copy code

```
<%@ page import="java.util.*, com.project.dao.UserDAO, com.project.model.U

<h2>User List</h2>
<a href="add-user.jsp">Add User</a>
<br><br>

<table border="1">
<tr>
    <th>ID</th>
    <th>Name</th>
    <th>Email</th>
    <th>Country</th>
    <th>Actions</th>
</tr>

<%
UserDAO dao = new UserDAO();
List<User> list = dao.getAllUsers();

for(User u : list) {
%>
<tr>
    <td><%= u.getId() %></td>
    <td><%= u.getName() %></td>
    <td><%= u.getEmail() %></td>
    <td><%= u.getCountry() %></td>
    <td>
        <a href="editUser?id=<%= u.getId() %>">Edit</a> |
        <a href="deleteUser?id=<%= u.getId() %>">Delete</a>
    </td>
</tr>
```

<% } %>